

# **REFACTORING DOCUMENT**

## **POTENTIAL REFACTORING TARGETS**

In order to identify potential refactoring targets we have chosen portions of the codebase which we have deemed to be the most important in the sense that it has a major role in the functioning of our project.

### **List of potential refactoring targets:**

GameEngine start\_game(),  
Player issue\_order(),  
MapEditor readMap(),  
GameEngine addPlayer(),  
Order execute(),  
Player class structure,  
MapEditor editMap(),  
WarMap edit functions,  
MapEditor saveMap(),  
MainMenu loadMap()/StartUp loadMap()  
Postload editMap(),  
Postload saveMap(),  
OrderExecution phase,  
IssueOrder phase,  
MapEditor showAllMaps().

## ACTUAL REFACTORING TARGETS

In order to choose the following refactoring targets, we have chosen functions which by refactoring allowed us to most easily implement the requirements of build 3 as some of these functions would not allow us to easily implement the requirements given prior to the refactoring.

### List of actual refactoring targets:

GameEngine start\_game(),  
Player issue\_order (),  
GameEngine addPlayer(),  
MapEditor readMap()/editMap(),  
MainMenu loadMap()/StartUp loadMap()

# Refactoring Operations

## Refactoring operation #1 (GameEngine start\_game())

The tests that apply to this class include:

testAssignCountries,  
testAddPlayer,  
testRemovePlayer,  
testGetNumOfReinforcements,  
testEditPhaseTransition,  
testPlayPhaseTransition,  
testStartUpValidation.

The refactoring operation was necessary after the introduction of Tournaments in build 3. To handle the tournaments more commands needed to be accepted and processed.

Apart from the tournament mode. In single player mode the introduction of Behavior strategies that do not involve human input meant that we needed to modify this method such that if a non-human behavior player is present then we do not take its order manually and auto perform all the required executions as well.

## Refactoring operation #2 (Player issue\_order ())

The tests that apply to this class include:

testNextOrderEmptyList,  
testNextOrder,  
testInvalidCountry

The introduction of behavior strategies meant that all of the order issuing was dependent on the behavior strategy that was present. So the main motive of the refactoring was to implement Strategy Pattern to pass the control of issuing order to the respective behavior that the player was initialized with.

Prior to refactoring all the issue order logic was handled by Player class itself as there was only human behavior. So, the previous implementation was moved to HumanBehaviorStrategy and the player class is only responsible for passing the issue order control to appropriate behavior.

### Refactoring operation #3 (GameEngine addPlayer())

The tests that apply to this class include:

testAssignCountries,  
testAddPlayer,  
testRemovePlayer,  
testGetNumOfReinforcements,  
testEditPhaseTransition,  
testPlayPhaseTransition,  
testStartUpValidation.

On the introduction of player behavior strategies, there arose the need to assign each player a certain behavior. The addPlayer before just created a player using the player's name as an input.

Now after refactoring this method is also responsible to assign the behavior to each player. So, there are two overloaded methods of addPlayer created. One, that can take the behavior strategy as a parameter, and the other that prompts the user to input the strategy and then it assigns it accordingly.

## Refactoring operation #4 (MapEditor readMap()/editMap())

The tests that apply to this class include:

testEditPhaseTransition,

testStartUpValidations,

WarMapTest class also indirectly tests as it will fail if readMap() fails.

These two functions edited the game engine's current WarMap by passing the game engine's WarMap object to the functions, this was incompatible with the implementation of the adapter pattern therefore were required to be changed.

Now after refactoring these methods both readMap() and editMap() do not take a WarMap as input and instead return a WarMap object, this allows for an Adapter class to return a WarMap object using the adaptee's readMap and editMap functions allowing it to implement the Adapter pattern.

## Refactoring operation #5 (MainMenu loadMap()/StartUp loadMap())

The tests that apply to this class include:

testEditPhaseTransition,

testPlayPhaseTransition

testStart Validations.

These functions were originally only able to load domination map files, and attempting to load a conquest map file type would not work, furthermore the refactored readMap() function of the MapEditor did not work with the old implementation of loadMap() so refactoring was necessary.

Now after refactoring the loadMap() method is able to load from both domination and conquest type maps using either the original map editor or the conquest map editor depending on the specified map file to be loaded. It was also refactored to use the new method signature of readMap().